A Tour of Go

Complete study guide

The Go Tour, selected Effective Go topics, and Go Concurrency Patterns: Context

Contents

Part I - More types: structs, slices, and maps

All 27 screens from the Tour, now flowed continuously rather than one screen per page.

Open the official source

Pointers

A pointer stores the address of a value. The type `*T` points to a value of type `T`, and its zero value is `nil`.

Use `&x` to take the address of `x`. Use `*p` to read or change the value reached through pointer `p`. Go deliberately omits pointer arithmetic.

Practice in the Go Tour

Example

```go
package main

import "fmt"

func main() {
    i, j := 42, 2701
    p := &i
    fmt.Println(*p)
    *p = 21
    fmt.Println(i)
    p = &j
    *p = *p / 37
    fmt.Println(j)
}
```

Structs

A `struct` groups named fields into one value. Each field has its own type, making structs the usual way to model related data in Go.

Example

```go
package main

import "fmt"

type Vertex struct {
    X int
    Y int
}

func main() {
    fmt.Println(Vertex{1, 2})
}
```

Struct Fields

Select or update a struct field with dot notation, such as `v.X`. Field names beginning with an uppercase letter are exported from their package.

Example

```go
package main

import "fmt"

type Vertex struct {
    X int
    Y int
}

func main() {
    v := Vertex{1, 2}
    v.X = 4
    fmt.Println(v.X)
}
```

Pointers to Structs

A pointer can refer to a struct. Go lets you write `p.X` instead of the more explicit `(*p).X`; the dereference is applied automatically for field selection.

Example

```go
package main

import "fmt"

type Vertex struct {
    X int
    Y int
}

func main() {
    v := Vertex{1, 2}
    p := &v
    p.X = 1e9
    fmt.Println(v)
}
```

Struct Literals

A struct literal creates a value directly. Positional literals list every field in order; keyed literals such as `Vertex{X: 1}` name selected fields and leave the rest at their zero values.

Prefix a literal with `&` to obtain a pointer to the new struct value.

Practice in the Go Tour

Example

```go
package main

import "fmt"

type Vertex struct { X, Y int }

var (
    v1 = Vertex{1, 2}
    v2 = Vertex{X: 1}
    v3 = Vertex{}
    p  = &Vertex{1, 2}
)

func main() {
    fmt.Println(v1, p, v2, v3)
}
```

Arrays

The type `[n]T` is an array of exactly `n` values of type `T`. The length is part of the type, so arrays have a fixed size and arrays of different lengths have different types.

Example

```go
package main

import "fmt"

func main() {
    var a [2]string
    a[0] = "Hello"
    a[1] = "World"
    fmt.Println(a[0], a[1])
    fmt.Println(a)

    primes := [6]int{2, 3, 5, 7, 11, 13}
    fmt.Println(primes)
}
```

Slices

A slice is a flexible view of a contiguous part of an array. Its type is `[]T`. The expression `a[low:high]` selects a half-open range: it includes `low` and excludes `high`.

Slices are used far more often than arrays because their length can change while their element type stays the same.

Practice in the Go Tour

Example

```go
package main

import "fmt"

func main() {
    primes := [6]int{2, 3, 5, 7, 11, 13}
    var s []int = primes[1:4]
    fmt.Println(s)
}
```

Slices Are References to Arrays

A slice descriptor refers to an underlying array; it does not hold an independent copy of the elements. Mutating an element through one slice is visible through every slice sharing that array.

Example

```go
package main

import "fmt"

func main() {
    names := [4]string{"John", "Paul", "George", "Ringo"}
    a := names[0:2]
    b := names[1:3]
    fmt.Println(a, b)

    b[0] = "XXX"
    fmt.Println(a, b)
    fmt.Println(names)
}
```

Slice Literals

A slice literal looks like an array literal with no length. For example, `[]bool{true, false}` creates an array and then returns a slice that refers to it.

The element type can itself be an unnamed struct, which is useful for compact test data and local tables.

Example

```go
package main

import "fmt"

func main() {
    q := []int{2, 3, 5, 7, 11, 13}
    r := []bool{true, false, true, true, false, true}
    s := []struct {
        i int
        b bool
    }{
        {2, true}, {3, false}, {5, true},
        {7, true}, {11, false}, {13, true},
    }
    fmt.Println(q, r, s)
}
```

Slice Defaults

Either bound may be omitted. The default low bound is zero; the default high bound is the current slice length. Thus `a[0:len(a)]`, `a[:len(a)]`, `a[0:]`, and `a[:]` select the same range.

Example

```
package main

import "fmt"

func main() {
    s := []int{2, 3, 5, 7, 11, 13}
    s = s[1:4]
    fmt.Println(s)
    s = s[:2]
    fmt.Println(s)
    s = s[1:]
    fmt.Println(s)
}
```

Slice Length and Capacity

A slice's length, `len(s)`, is the number of accessible elements. Its capacity, `cap(s)`, is the number of elements available in the backing array from the slice's first element onward.

Re-slicing can grow the length only up to the capacity. Trying to select past the capacity causes a run-time panic.

Practice in the Go Tour

Example

```
package main

import "fmt"

func main() {
    s := []int{2, 3, 5, 7, 11, 13}
    printSlice(s)
    s = s[:0]
    printSlice(s)
    s = s[:4]
    printSlice(s)
    s = s[2:]
    printSlice(s)
}

func printSlice(s []int) {
    fmt.Printf("len=%d cap=%d %v\n", len(s), cap(s), s)
}
```

Nil Slices

The zero value of a slice is `nil`. A nil slice has length and capacity zero and no backing array. It is safe to range over or append to a nil slice.

Example

```
package main

import "fmt"

func main() {
    var s []int
    fmt.Println(s, len(s), cap(s))
    if s == nil {
        fmt.Println("nil!")
    }
}
```

Creating a Slice with make

The built-in `make` allocates a zeroed backing array and returns a slice. Use `make([]T, length)` or add a third argument for a distinct capacity: `make([]T, length, capacity)`.

Example

```
package main

import "fmt"

func main() {
    a := make([]int, 5)
    printSlice("a", a)
    b := make([]int, 0, 5)
    printSlice("b", b)
    c := b[:2]
    printSlice("c", c)
    d := c[2:5]
    printSlice("d", d)
}

func printSlice(label string, x []int) {
    fmt.Printf("%s len=%d cap=%d %v\n",
        label, len(x), cap(x), x)
}
```

Slices of Slices

A slice can contain values of any type, including other slices. A `[][]string` is therefore a slice whose elements are independently sized `[]string` values.

Example

```go
package main

import (
    "fmt"
    "strings"
)

func main() {
    board := [][]string{
        {"_", "_", "_"},
        {"_", "_", "_"},
        {"_", "_", "_"},
    }
    board[0][0], board[2][2] = "X", "O"
    board[1][2], board[1][0] = "X", "O"
    board[0][2] = "X"
    for i := 0; i < len(board); i++ {
        fmt.Println(strings.Join(board[i], " "))
    }
}
```

Appending to a Slice

The built-in `append` returns a slice containing the old elements followed by new ones. Always use the returned slice: when capacity is insufficient, Go allocates a larger backing array.

Its conceptual signature is `func append(s []T, values ...T) []T`.

> **Practice in the Go Tour**

Example

```go
package main

import "fmt"

func main() {
    var s []int
    printSlice(s)
    s = append(s, 0)
    printSlice(s)
    s = append(s, 1)
    printSlice(s)
    s = append(s, 2, 3, 4)
    printSlice(s)
}

func printSlice(s []int) {
    fmt.Printf("len=%d cap=%d %v\n", len(s), cap(s), s)
}
```

Range

A `for ... range` loop iterates over a slice or map. For a slice, each iteration produces the index and a copy of the element at that index.

Example

```
package main

import "fmt"

var pow = []int{1, 2, 4, 8, 16, 32, 64, 128}

func main() {
    for i, v := range pow {
        fmt.Printf("2**%d = %d\n", i, v)
    }
}
```

Range Continued

Assign an unwanted result to the blank identifier _. If only the index is needed, omit the second variable entirely: `for i := range values`.

Example

```
package main

import "fmt"

func main() {
    pow := make([]int, 10)
    for i := range pow {
        pow[i] = 1 << uint(i)
    }
    for _, value := range pow {
        fmt.Printf("%d\n", value)
    }
}
```

Exercise: Slices

Implement `Pic` so it returns `dy` rows, each containing `dx` unsigned 8-bit values. Allocate every inner slice, then fill it with a function of `x` and `y` such as `(x+y)/2`, `x*y`, or `x^y`.

Practice in the Go Tour

Example

```
package main

import "golang.org/x/tour/pic"

func Pic(dx, dy int) [][]uint8 {
    // Allocate rows and fill pixels here.
}

func main() {
    pic.Show(Pic)
}
```

Maps

A map associates keys with values. The type `map[K]V` maps keys of type `K` to values of type `V`. A nil map can be read but not written; use `make` or a map literal before inserting entries.

Example

```go
package main

import "fmt"

type Vertex struct { Lat, Long float64 }
var m map[string]Vertex

func main() {
    m = make(map[string]Vertex)
    m["Bell Labs"] = Vertex{40.68433, -74.39967}
    fmt.Println(m["Bell Labs"])
}
```

Map Literals

A map literal initializes both the map and its entries. Every element supplies a key followed by the corresponding value.

Example

```go
package main

import "fmt"

type Vertex struct { Lat, Long float64 }

var m = map[string]Vertex{
    "Bell Labs": Vertex{40.68433, -74.39967},
    "Google":    Vertex{37.42202, -122.08408},
}

func main() {
    fmt.Println(m)
}
```

Map Literals Continued

When a map literal's value type is a named type, the type name may be omitted from each element. The shorter `"Google": {37.42, -122.08}` form is equivalent to repeating `Vertex`.

Example

```go
package main

import "fmt"

type Vertex struct { Lat, Long float64 }

var m = map[string]Vertex{
    "Bell Labs": {40.68433, -74.39967},
    "Google":    {37.42202, -122.08408},
}

func main() {
    fmt.Println(m)
}
```

Mutating Maps

Insert or update with `m[key] = value`; retrieve with `value = m[key]`; delete with `delete(m, key)`. The two-result lookup `value, ok := m[key]` distinguishes a missing key from a present key holding the zero value.

Practice in the Go Tour

Example

```go
package main

import "fmt"

func main() {
    m := make(map[string]int)
    m["Answer"] = 42
    fmt.Println("The value:", m["Answer"])
    m["Answer"] = 48
    fmt.Println("The value:", m["Answer"])
    delete(m, "Answer")
    v, ok := m["Answer"]
    fmt.Println("The value:", v, "Present?", ok)
}
```

Exercise: Maps

Implement `WordCount` so it returns the frequency of every word in `s`. A practical approach is to split with `strings.Fields`, create a `map[string]int`, and increment one entry per word.

Practice in the Go Tour

Example

```go
package main

import "golang.org/x/tour/wc"

func WordCount(s string) map[string]int {
    // Split s, count each word, and return the map.
    return map[string]int{"x": 1}
}

func main() {
    wc.Test(WordCount)
}
```

Function Values

Functions are values. They can be stored in variables, passed as arguments, and returned from other functions. A function's full parameter and result signature is its type.

Example

```go
package main

import (
    "fmt"
    "math"
)

func compute(fn func(float64, float64) float64) float64 {
    return fn(3, 4)
}

func main() {
    hypot := func(x, y float64) float64 {
        return math.Sqrt(x*x + y*y)
    }
    fmt.Println(hypot(5, 12))
    fmt.Println(compute(hypot))
    fmt.Println(compute(math.Pow))
}
```

Function Closures

A closure is a function value that refers to variables outside its own body. The captured variables remain available between calls, and different closure instances can hold independent state.

Example

```go
package main

import "fmt"

func adder() func(int) int {
    sum := 0
    return func(x int) int {
        sum += x
        return sum
    }
}

func main() {
    pos, neg := adder(), adder()
    for i := 0; i < 10; i++ {
        fmt.Println(pos(i), neg(-2*i))
    }
}
```

Exercise: Fibonacci Closure

Implement `fibonacci` so it returns a closure that emits successive Fibonacci numbers: `0, 1, 1, 2, 3, 5, ...`. Keep the two most recent values in variables captured by the returned function.

Practice in the Go Tour

Example

```go
package main

import "fmt"

func fibonacci() func() int {
    // Return a closure with private sequence state.
}

func main() {
    f := fibonacci()
    for i := 0; i < 10; i++ {
        fmt.Println(f())
    }
}
```

Congratulations!

You have completed the More types chapter. You can now model data with structs and maps, work with shared array storage through slices, and treat functions as first-class values with persistent closure state.

The next chapter in the requested guide will cover methods and interfaces.

Part I - Methods and interfaces

All 26 screens covering receiver methods, interfaces, standard protocols, and exercises.

Open the official source

Methods

Go has no classes, but methods can be defined on named types. A method is a function with a receiver between `func` and the method name.

Practice in the Go Tour

Example

```
type Vertex struct { X, Y float64 }

func (v Vertex) Abs() float64 {
    return math.Sqrt(v.X*v.X + v.Y*v.Y)
}

v := Vertex{3, 4}
fmt.Println(v.Abs())
```

Methods Are Functions

A method is an ordinary function with a distinguished receiver argument. Rewriting a method as a function preserves the behavior but changes the call syntax.

Example

```
func Abs(v Vertex) float64 {
    return math.Sqrt(v.X*v.X + v.Y*v.Y)
}

fmt.Println(Abs(Vertex{3, 4}))
```

Methods Continued

Receivers need not be structs. The receiver's named type must be defined in the same package as the method.

Example

```
type MyFloat float64

func (f MyFloat) Abs() float64 {
    if f < 0 { return float64(-f) }
    return float64(f)
}
```

Pointer Receivers

A method with receiver `*T` can modify the original value. Pointer receivers are common when methods mutate state or when copying a large value would be wasteful.

Practice in the Go Tour

Example

```
func (v *Vertex) Scale(f float64) {
    v.X *= f
    v.Y *= f
}

v := Vertex{3, 4}
v.Scale(10)
```

Pointers and Functions

Regular functions require the caller to pass a pointer explicitly when mutation is intended. Value arguments are copied.

Example

```
func Scale(v *Vertex, f float64) {
    v.X *= f
    v.Y *= f
}

v := Vertex{3, 4}
Scale(&v, 10)
```

Methods and Pointer Indirection

For an addressable value `v`, Go interprets `v.Scale(5)` as `(&v).Scale(5)` when `Scale` has a pointer receiver.

Example

```
v := Vertex{3, 4}
v.Scale(2)       // compiler uses (&v).Scale(2)
ScaleFunc(&v, 10)

p := &Vertex{4, 3}
p.Scale(3)
ScaleFunc(p, 8)
```

Methods and Pointer Indirection (2)

The reverse convenience also applies: a pointer may call a value-receiver method, so `p.Abs()` is interpreted as `(*p).Abs()`.

Example

```
v := Vertex{3, 4}
fmt.Println(v.Abs())

p := &Vertex{4, 3}
fmt.Println(p.Abs())
fmt.Println(AbsFunc(*p))
```

Choosing a Value or Pointer Receiver

Use a pointer receiver to modify the receiver or avoid copying it. Keep receiver choice consistent across a type unless there is a strong reason not to.

Example

```
func (v *Vertex) Scale(f float64) { v.X *= f; v.Y *= f }
func (v *Vertex) Abs() float64 {
    return math.Sqrt(v.X*v.X + v.Y*v.Y)
}
```

Interfaces

An interface is a set of method signatures. A value implements the interface when its method set contains those methods.

Practice in the Go Tour

Example

```
type Abser interface { Abs() float64 }

var a Abser
a = MyFloat(-math.Sqrt2)
v := Vertex{3, 4}
a = &v            // *Vertex implements Abser
// a = v           // Vertex does not
```

Interfaces Are Implemented Implicitly

There is no implements declaration. This implicit satisfaction lets interfaces live near their consumers and decouples implementations from interface definitions.

Example

```
type I interface { M() }
type T struct { S string }

func (t T) M() { fmt.Println(t.S) }

var i I = T{"hello"}
i.M()
```

Interface Values

Conceptually, an interface value contains an underlying concrete value and its concrete type. Calling a method dispatches to that concrete type's implementation.

Example

```
var i I
i = &T{"Hello"}
describe(i) // (&{Hello}, *main.T)
i.M()

i = F(math.Pi)
describe(i) // (3.14159..., main.F)
i.M()
```

Interface Values with Nil Underlying Values

An interface holding a typed nil pointer is itself non-nil. Its method still runs with a nil receiver, which the implementation may handle explicitly.

Example

```
func (t *T) M() {
    if t == nil { fmt.Println("<nil>"); return }
    fmt.Println(t.S)
}

var t *T
var i I = t
i.M()
```

Nil Interface Values

A nil interface contains neither a value nor a concrete type. Calling a method on it panics because there is no concrete method to dispatch to.

Example

```
var i I
fmt.Printf("(%v, %T)
", i, i)
// i.M() // run-time panic
```

The Empty Interface

The empty interface `interface{}` has no methods and can hold any value. Modern Go usually spells this alias as `any`.

Example

```
var i any
i = 42
fmt.Printf("(%v, %T)
", i, i)
i = "hello"
fmt.Printf("(%v, %T)
", i, i)
```

Type Assertions

A type assertion `i.(T)` extracts an underlying `T`. The two-result form returns a value and success flag instead of panicking on mismatch.

Practice in the Go Tour

Example

```
var i any = "hello"
s := i.(string)
s, ok := i.(string)
f, ok := i.(float64)
fmt.Println(s, ok, f)
```

Type Switches

A type switch uses `i.(type)` and selects a branch based on the interface value's concrete type.

Example

```
func describe(i any) {
    switch v := i.(type) {
    case int:
        fmt.Println("twice:", v*2)
    case string:
        fmt.Println("bytes:", len(v))
    default:
        fmt.Printf("unknown %T
", v)
    }
}
```

Stringers

Types implementing `fmt.Stringer` control their default textual form through `String() string`.

Example

```
type Person struct { Name string; Age int }

func (p Person) String() string {
    return fmt.Sprintf("%v (%v years)", p.Name, p.Age)
}

fmt.Println(Person{"Arthur Dent", 42})
```

Exercise: Stringers

Add a String method to IPAddr so the four bytes print as a dotted IPv4 address.

Practice in the Go Tour

Starter code

```
type IPAddr [4]byte

// TODO: func (ip IPAddr) String() string

hosts := map[string]IPAddr{
    "loopback": {127, 0, 0, 1},
    "googleDNS": {8, 8, 8, 8},
}
```

Errors

Go represents failures with values implementing `error`. A nil error means success; a non-nil value can carry structured context.

Example

```
type MyError struct { When time.Time; What string }

func (e *MyError) Error() string {
    return fmt.Sprintf("at %v, %s", e.When, e.What)
}

func run() error {
    return &MyError{time.Now(), "it didn't work"}
}
```

Why use this:
if err := run(); err != nil {
fmt.Println(err)
}
instead of two lines?You could write it as:
err := run()
if err != nil { fmt.Println(err)}
Same behavior.
But the one-liner has a nice side effect: err's scope is limited to the if block (and any else attached to it). Once you leave the if, err no longer exists. This is idiomatic Go — it avoids cluttering the outer scope

Exercise: Errors

Define ErrNegativeSqrt, give it an Error method, and make Sqrt return it for negative inputs.

Practice in the Go Tour

Starter code

```
type ErrNegativeSqrt float64

// TODO: implement Error() string.
func Sqrt(x float64) (float64, error) {
    // Return ErrNegativeSqrt(x) when x < 0.
    return 0, nil
}
```

Readers

The `io.Reader` interface exposes `Read([]byte) (int, error)`. Readers return populated bytes and commonly signal end-of-stream with `io.EOF`.

Example

```
r := strings.NewReader("Hello, Reader!")
b := make([]byte, 8)
for {
    n, err := r.Read(b)
    fmt.Printf("n=%v err=%v %q
", n, err, b[:n])
    if err == io.EOF { break }
}
```

Exercise: Readers

Implement a Reader that emits an infinite stream of ASCII 'A' bytes.

Practice in the Go Tour

Starter code

```
type MyReader struct{}

// TODO: Add
// Read([]byte) (int, error)

reader.Validate(MyReader{})
```

Exercise: rot13Reader

Wrap another io.Reader and transform alphabetical bytes with ROT13 as they are read.

Practice in the Go Tour

Starter code

```
type rot13Reader struct { r io.Reader }

// TODO: implement Read(p []byte) (int, error).

s := strings.NewReader("Lbh penpxrq gur pbqr!")
r := rot13Reader{s}
io.Copy(os.Stdout, &r)
```

Images

The `image.Image` interface requires `ColorModel`, `Bounds`, and `At`. Standard image types implement it directly.

Example

```
m := image.NewRGBA(image.Rect(0, 0, 100, 100))
fmt.Println(m.Bounds())
fmt.Println(m.At(0, 0).RGBA())
```

Exercise: Images

Define an Image type and implement image.Image so the Tour can display a generated picture.

Practice in the Go Tour

Starter code

```
type Image struct{}

// TODO: ColorModel() color.Model
// TODO: Bounds() image.Rectangle
// TODO: At(x, y int) color.Color

pic.ShowImage(Image{})
```

Congratulations!

You have completed methods and interfaces: receiver choices, implicit interfaces, dynamic interface values, standard protocols, and error values.

Practice in the Go Tour

Part I - Generics

All 3 screens covering generic functions and generic types.

Open the official source

Type Parameters

Type parameters appear in brackets before ordinary parameters. The `comparable` constraint permits equality operations for values of type `T`.

Practice in the Go Tour

Example

```
func Index[T comparable](s []T, x T) int {
    for i, v := range s {
        if v == x { return i }
    }
    return -1
}

fmt.Println(Index([]int{10, 20, 15}, 15))
fmt.Println(Index([]string{"foo", "bar"}, "bar"))
```

Generic Types

Named types may also have type parameters. The constraint `any` accepts every type and is useful for reusable containers.

Practice in the Go Tour

Example

```
type List[T any] struct {
    next *List[T]
    val  T
}

words := &List[string]{val: "gopher"}
numbers := &List[int]{val: 42}
_, _ = words, numbers
```

Congratulations!

You have covered generic functions, constraints, type inference, and parameterized data structures.

Practice in the Go Tour

Part I - Concurrency

All 11 screens covering goroutines, channels, select, mutexes, and exercises.

Open the official source

Goroutines

A goroutine is a lightweight concurrent function execution managed by the Go runtime. Prefix a call with go to start it.

Practice in the Go Tour

Example

```
func say(s string) {
    for i := 0; i < 5; i++ {
        time.Sleep(100 * time.Millisecond)
        fmt.Println(s)
    }
}

go say("world")
say("hello")
```

Channels

Channels are typed conduits. Sends and receives use <- and block until communication can proceed, providing synchronization as well as data transfer.

Example

```
func sum(s []int, c chan int) {
    total := 0
    for _, v := range s { total += v }
    c <- total
}

c := make(chan int)
go sum(values[:3], c)
go sum(values[3:], c)
x, y := <-c, <-c
```

Buffered Channels

A buffered channel accepts sends until its buffer is full. Receives block only while the buffer is empty.

Example

```
ch := make(chan int, 2)
ch <- 1
ch <- 2
fmt.Println(<-ch)
fmt.Println(<-ch)
```

Range and Close

A sender may close a channel to announce that no more values will arrive. Receivers can use the comma-ok form or range until closure.

Practice in the Go Tour

Example

```
func fibonacci(n int, c chan int) {
    x, y := 0, 1
    for i := 0; i < n; i++ { c <- x; x, y = y, x+y }
    close(c)
}

for value := range c { fmt.Println(value) }
```

Select

A `select` waits on multiple channel operations and runs one ready case. If several cases are ready, one is chosen pseudo-randomly.

Example

```
select {
case c <- x:
    x, y = y, x+y
case <-quit:
    fmt.Println("quit")
    return
}
```

Default Selection

A `default` case makes a select non-blocking when no communication case is ready.

Practice in the Go Tour

Example

```
select {
case <-tick:
    fmt.Println("tick")
case <-boom:
    fmt.Println("BOOM!")
    return
default:
    time.Sleep(50 * time.Millisecond)
}
```

Exercise: Equivalent Binary Trees - Concept

Different binary tree shapes may contain the same ordered sequence. A channel-based walker can expose the sequence independently of shape.

Practice in the Go Tour

Example

```
type Tree struct {
    Left  *Tree
    Value int
    Right *Tree
}

// In-order traversal produces sorted values.
```

Exercise: Equivalent Binary Trees - Implementation

Implement Walk to stream values, then Same to compare two streams from concurrently walked trees.

Practice in the Go Tour

Starter code

```
func Walk(t *tree.Tree, ch chan int) {
    // TODO: in-order traversal; close ch when done.
}

func Same(t1, t2 *tree.Tree) bool {
    // TODO: compare the two value streams.
    return false
}
```

sync.Mutex

Use a mutex when protecting shared state directly is clearer than moving ownership through channels. Lock every access to the protected value.

Example

```
type SafeCounter struct {
    mu sync.Mutex
    v  map[string]int
}

func (c *SafeCounter) Inc(key string) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.v[key]++
}
```

Exercise: Web Crawler

Parallelize Crawl while ensuring each URL is fetched at most once. Protect the visited set with a mutex or own it in a coordinating goroutine.

Practice in the Go Tour

Starter code

```
func Crawl(url string, depth int, fetcher Fetcher) {
    if depth <= 0 { return }
    body, urls, err := fetcher.Fetch(url)
    if err != nil { fmt.Println(err); return }
    fmt.Printf("found: %s %q
", url, body)
    for _, u := range urls {
        // TODO: crawl concurrently without duplicates.
        Crawl(u, depth-1, fetcher)
    }
}
```

Where to Go from Here

Continue with package documentation, the language specification, Go concurrency talks, code walks, and the Go Blog. The links below point back to the official Tour resources.

Practice in the Go Tour

Go Routines vs C# Async Tasks

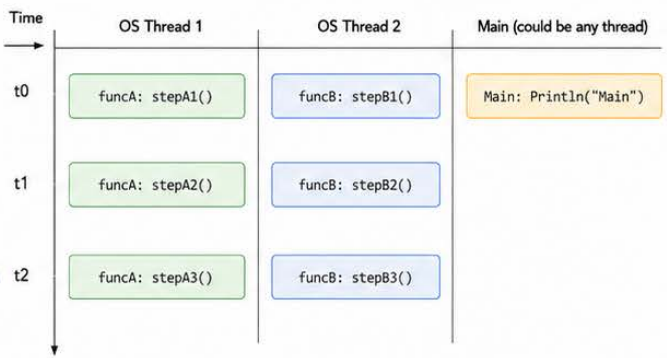
Go Routines (Parallel Execution)

Go routines run concurrently and can execute in parallel on multiple OS threads. The scheduler runs their steps at the same time.

Example Code

```
go funcA() {                go funcB() {
    stepA1()                    stepB1()
    stepA2()                    stepB2()
    stepA3()                    stepB3()
}()                         }()              fmt.Println("Main")
```

Execution Timeline (Parallel Execution)



Key Point

Go routines are scheduled to run in parallel.

Steps from different go routines can execute in parallel (at the same time) on different OS threads/cores. The order between them is not guaranteed.

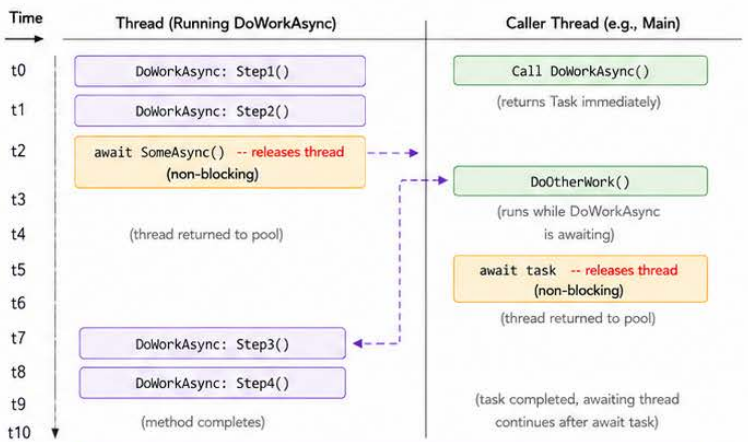
C# Async Tasks (Sequential Until Await)

Async methods run synchronously (line by line) on the current thread until they hit an await (non-blocking point). At that point, the thread is released back to the caller. It resumes only after the awaited operation completes.

Example Code

```
async Task DoWorkAsync() {
    Step1();            // runs synchronously
    Step2();            // runs synchronously
    await SomeAsync(); // non-blocking point
    Step3();            // runs after await completes
    Step4();
}
```

```
var task = DoWorkAsync();
// Thread is free here
DoOtherWork();
await task; // waits for DoWorkAsync
            to complete
```



Key Point

Async methods run sequentially until the first await.

At await, the thread is released back to the caller. The caller can do other work. When the awaited operation completes, the async method resumes from the next line.

Summary

Go Routines = Concurrent (parallel, nondeterministic order)

Good for CPU-bound and I/O-bound concurrent workloads.

C# Async Tasks = Sequential until await (cooperative multitasking)

Good for I/O-bound operations and keeping threads free.

Part II - Effective Go: Formatting

The requested Formatting topic. Effective Go remains useful for core idioms but is not actively updated for every modern language and ecosystem feature.

Open the official source

Formatting with gofmt

Let `gofmt` or `go fmt` determine indentation and alignment. Standard formatting makes layout predictable and removes style debates.

Example

```
type T struct {
    name  string // name of the object
    value int    // its value
}
```

Indentation, Line Length, and Parentheses

Use tabs for indentation, wrap a line when it feels too long, and omit parentheses around `if`, `for`, and `switch` conditions.

Example

```
if x > 0 {
    return y
}

for i := 0; i < 10; i++ {
    sum += i
}
```

Part II - Effective Go: Functions

Multiple returns, named results, and defer.

Open the official source

Multiple Return Values

Return a result and an error together instead of encoding failure inside a normal value or mutating output arguments.

Example

```
func (file *File) Write(b []byte) (n int, err error)

n, err := file.Write(data)
if err != nil { return err }
if n != len(data) { return io.ErrShortWrite }
```

Named Result Parameters

Named results document the meaning of same-typed return values and may support a bare return. Use them when they clarify the function, not merely to save typing.

Example

```
func ReadFull(r Reader, buf []byte) (n int, err error) {
    for len(buf) > 0 && err == nil {
        var nr int
        nr, err = r.Read(buf)
        n += nr
        buf = buf[nr:]
    }
    return
}
```

Defer

defer schedules cleanup for function return. Its arguments are evaluated immediately; deferred calls execute in last-in, first-out order.

Example

```
func Contents(name string) (string, error) {
    f, err := os.Open(name)
    if err != nil { return "", err }
    defer f.Close()
    data, err := io.ReadAll(f)
    return string(data), err
}
```

Part II - Effective Go: Data

Allocation, composite literals, arrays, slices, maps, formatting, and append.

Open the official source

Allocation with new

`new(T)` returns a pointer to a newly allocated zero value. Design types so their zero value is useful whenever practical.

Example

```
type SyncedBuffer struct {
    lock   sync.Mutex
    buffer bytes.Buffer
}

p := new(SyncedBuffer)
var v SyncedBuffer
_, _ = p, v
```

Constructors and Composite Literals

When initialization is needed, use a constructor and keyed composite literals. Returning the address of a local value is safe in Go.

Example

```
func NewFile(fd int, name string) *File {
    if fd < 0 { return nil }
    return &File{fd: fd, name: name}
}
```

Allocation with make

`make` initializes slices, maps, and channels and returns a value of that type. Unlike `new`, it does not return a pointer.

Example

```
v := make([]int, 100)
m := make(map[string]int)
c := make(chan int, 8)
```

Arrays

Arrays are values: assignment and parameter passing copy all elements, and the length is part of the type. Prefer slices for most sequence APIs.

Example

```
func Sum(a *[3]float64) (sum float64) {
    for _, v := range *a { sum += v }
    return
}

array := [...]float64{7.0, 8.5, 9.1}
x := Sum(&array)
```

Slices

Slices refer to underlying arrays. Element mutations are visible to the caller, but the slice descriptor itself is passed by value, so growth must be returned.

Example

```
func Append(s, data []byte) []byte {
    n := len(s)
    if n+len(data) > cap(s) {
        grown := make([]byte, (n+len(data))*2)
        copy(grown, s)
        s = grown
    }
    s = s[:n+len(data)]
    copy(s[n:], data)
    return s
}
```

Two-Dimensional Slices

A two-dimensional slice is a slice of slices. Allocate rows independently when lengths may vary, or divide one backing allocation for compact fixed-width data.

Example

```
picture := make([][]uint8, ySize)
pixels := make([]uint8, xSize*ySize)
for i := range picture {
    picture[i], pixels = pixels[:xSize], pixels[xSize:]
}
```

Maps

Map keys must support equality. Missing lookups return the element zero value; use the comma-ok form when presence matters.

Example

```
seconds, ok := timeZone[tz]
if !ok {
    log.Println("unknown time zone:", tz)
}
delete(timeZone, "PDT")
```

Printing

The `fmt` package formats values by type. Useful verbs include `%v`, `%+v`, `%#v`, `%q`, and `%T`. A String method defines the default text form.

Example

```
fmt.Printf("%v
", value)
fmt.Printf("%+v
", value)
fmt.Printf("%#v
", value)
fmt.Printf("%T
", value)
```

Append

Always use the slice returned by `append`. Add another slice with `...` so its elements become variadic arguments.

Example

```
x := []int{1, 2, 3}
y := []int{4, 5, 6}
x = append(x, y...)
fmt.Println(x)
```

Part II - Effective Go: Interfaces

Interfaces, conversions, assertions, generality, and method adapters.

Open the official source

Interfaces

Small behavioral interfaces are idiomatic. A type may satisfy several interfaces and can expose distinct capabilities to different consumers.

Example

```
type Sequence []int

func (s Sequence) Len() int           { return len(s) }
func (s Sequence) Less(i, j int) bool { return s[i] < s[j] }
func (s Sequence) Swap(i, j int)      { s[i], s[j] = s[j], s[i] }
```

Conversions

Convert between types with compatible underlying representations to access another method set or reuse an existing implementation.

Example

```
func (s Sequence) String() string {
    copy := append(Sequence(nil), s...)
    sort.IntSlice(copy).Sort()
    return fmt.Sprint([]int(copy))
}
```

Interface Conversions and Type Assertions

Use a type switch for several possibilities and a comma-ok assertion for one safe extraction.

Example

```
switch v := value.(type) {
case string:
    return v
case fmt.Stringer:
    return v.String()
}

str, ok := value.(string)
```

Generality

When callers need only an interface, return that interface and keep the implementation unexported. Constructors can then swap algorithms without changing consumers.

Example

```
type Block interface {
    BlockSize() int
    Encrypt(dst, src []byte)
    Decrypt(dst, src []byte)
}

func NewCTR(block Block, iv []byte) Stream
```

Interfaces and Methods

Methods can be attached to structs, scalar named types, channels, and function types. Adapter types such as http.HandlerFunc turn an ordinary function into an interface implementation.

Example

```
type HandlerFunc func(http.ResponseWriter, *http.Request)

func (f HandlerFunc) ServeHTTP(
    w http.ResponseWriter,
    r *http.Request,
) {
    f(w, r)
}
```

Part II - Effective Go: Errors

Error values, panic, and recover.

Open the official source

Error Values

Return detailed error values with operational context. Callers may inspect richer error types when recovery depends on the cause.

Example

```
type PathError struct {
    Op   string
    Path string
    Err  error
}

func (e *PathError) Error() string {
    return e.Op + " " + e.Path + ": " + e.Err.Error()
}
```

Panic

Use panic only for genuinely unrecoverable or impossible conditions. Libraries should normally return errors so callers control policy.

Example

```
func init() {
    if os.Getenv("USER") == "" {
        panic("no value for USER")
    }
}
```

Recover

recover can stop panic unwinding only when called directly inside a deferred function. Use it at controlled boundaries, and convert internal panics back into ordinary errors.

Example

```
func safelyDo(work *Work) {
    defer func() {
        if value := recover(); value != nil {
            log.Println("work failed:", value)
        }
    }()
    do(work)
}
```

Part II - Effective Go: Concurrency

Communicating ownership, goroutines, channels, reply channels, parallelization, and buffer reuse.

Open the official source

Share by Communicating

Prefer transferring ownership through channels when it clarifies synchronization. Mutexes remain appropriate for simple shared counters and other tightly scoped state.

Goroutines

Prefix a call with go to execute it concurrently. Function literals are closures, so referenced variables remain alive while the goroutine uses them.

Example

```
func Announce(message string, delay time.Duration) {
    go func() {
        time.Sleep(delay)
        fmt.Println(message)
    }()
}
```

Channels

Unbuffered channels combine communication with synchronization. Buffered channels can also act as bounded semaphores or work queues.

Example

```
sem := make(chan struct{}, maxOutstanding)

sem <- struct{}{}
go func(req *Request) {
    defer func() { <-sem }()
    process(req)
}(req)
```

Channels of Channels

Because channels are first-class values, a request may carry its own reply channel, providing safe demultiplexing without shared response state.

Example

```
type Request struct {
    args       []int
    f          func([]int) int
    resultChan chan int
}

req := &Request{args, sum, make(chan int)}
clientRequests <- req
answer := <-req.resultChan
```

Parallelization

Concurrency structures work; parallelism runs independent work simultaneously. Partition only calculations that are truly independent, then wait for completion.

Example

```
done := make(chan struct{}, workers)
for i := 0; i < workers; i++ {
    go func(start, end int) {
        processRange(start, end)
        done <- struct{}{}
    }(bounds(i))
}
for i := 0; i < workers; i++ { <-done }
```

A Leaky Buffer

A buffered channel plus a non-blocking `select` can implement a reusable buffer pool. Allocate when empty and let excess buffers be garbage-collected when the pool is full.

Example

```
select {
case b = <-freeList:
default:
    b = new(Buffer)
}

select {
case freeList <- b:
default:
}
```

Part III - Go Concurrency Patterns: Context

A complete guided treatment of request-scoped deadlines, cancellation, values, HTTP propagation, and adaptation patterns.

Open the official source

Introduction

Server requests often fan out into several goroutines. They need shared deadlines, cancellation signals, and request-scoped values so abandoned work exits quickly.

The Context Contract

A `context.Context` carries deadline, cancellation, and scoped values across API boundaries. It is safe for simultaneous use by many goroutines.

Example

```
type Context interface {
    Deadline() (time.Time, bool)
    Done() <-chan struct{}
    Err() error
    Value(key any) any
}
```

Derived Contexts

Contexts form a tree. Canceling a parent cancels its descendants. Always call the returned `CancelFunc` to release timers and resources.

Example

```
ctx, cancel := context.WithTimeout(
    context.Background(),
    2*time.Second,
)
defer cancel()

select {
case <-workDone:
case <-ctx.Done():
    return ctx.Err()
}
```

Request Handler

Create a request context, apply an optional timeout, validate input, attach request-scoped data, and pass the context through every downstream call.

Example

```
ctx, cancel := context.WithCancel(req.Context())
if timeout > 0 {
    ctx, cancel = context.WithTimeout(ctx, timeout)
}
defer cancel()

results, err := search(ctx, query)
```

Typed Context Values

Hide context keys behind package functions. Use an unexported key type to prevent collisions, and store only request-scoped data.

Example

```
type key int
const userIPKey key = 0

func NewContext(ctx context.Context, ip net.IP) context.Context {
    return context.WithValue(ctx, userIPKey, ip)
}

func FromContext(ctx context.Context) (net.IP, bool) {
    ip, ok := ctx.Value(userIPKey).(net.IP)
    return ip, ok
}
```

Propagating Context to HTTP

Attach the context to outbound requests with `req.WithContext(ctx)`. The HTTP client then cancels in-flight work when the context finishes.

Example

```
req, err := http.NewRequest(http.MethodGet, url, nil)
if err != nil { return err }
req = req.WithContext(ctx)

resp, err := http.DefaultClient.Do(req)
if err != nil { return err }
defer resp.Body.Close()
```

Waiting for Work or Cancellation

Use `select` to return promptly when `ctx.Done()` closes. A buffered result channel prevents a worker from becoming stuck if the caller stops listening.

Example

```
result := make(chan error, 1)
go func() { result <- doWork() }()

select {
case <-ctx.Done():
    return ctx.Err()
case err := <-result:
    return err
}
```

Adapting Existing Code

Bridge framework-specific cancellation or request storage at the boundary, then expose the standard Context contract to the rest of the call graph.

Practical Rules

Pass `ctx context.Context` as the first parameter. Do not store contexts in structs unless an API specifically requires it.

Do not pass nil; use context.TODO when the correct parent is not yet known. Values should carry request-scoped metadata, not optional function parameters.

Example

```
func Lookup(ctx context.Context, key string) (Value, error) {
    // Check ctx.Done in long loops and blocking operations.
    return backend.Lookup(ctx, key)
}
```

Conclusion

A shared Context convention lets independently developed packages cooperate on cancellation, timeouts, and request metadata, reducing leaks and making service boundaries composable.

Sources and further practice

A Tour of Go - More types

A Tour of Go - Methods and interfaces

A Tour of Go - Generics

A Tour of Go - Concurrency

Effective Go

Go Concurrency Patterns: Context

Go Playground